The sequence `[()]` is legal, but `[(])` is wrong. Obviously, it is not worthwhile writing a huge program for this, but it turns out that it is easy to check these things. For simplicity, we will just check for balancing of parentheses, brackets, and braces and ignore any other character that appears.

The simple algorithm uses a stack and is as follows:

> Make an empty stack. Read characters until end of file. If the character is an opening symbol, push it onto the stack. If it is a closing symbol and the stack is empty, report an error. Otherwise, pop the stack. If the symbol popped is not the corresponding opening symbol, then report an error. At end of file, if the stack is not empty, report an error.

You should be able to convince yourself that this algorithm works. It is clearly linear and actually makes only one pass through the input. It is thus online and quite fast. Extra work can be done to attempt to decide what to do when an error is reported—such as identifying the likely cause.

Postfix Expressions

Suppose we have a pocket calculator and would like to compute the cost of a shopping trip. To do so, we add a list of numbers and multiply the result by 1.06; this computes the purchase price of some items with local sales tax added. If the items are 4.99, 5.99, and 6.99, then a natural way to enter this would be the sequence

$$4.99 + 5.99 + 6.99 * 1.06 =$$

Depending on the calculator, this produces either the intended answer, 19.05, or the scientific answer, 18.39. Most simple four-function calculators will give the first answer, but many advanced calculators know that multiplication has higher precedence than addition.

On the other hand, some items are taxable and some are not, so if only the first and last items were actually taxable, then the sequence

$$4.99 * 1.06 + 5.99 + 6.99 * 1.06 =$$

would give the correct answer (18.69) on a scientific calculator and the wrong answer (19.37) on a simple calculator. A scientific calculator generally comes with parentheses, so we can always get the right answer by parenthesizing, but with a simple calculator we need to remember intermediate results.

A typical evaluation sequence for this example might be to multiply 4.99 and 1.06, saving this answer as A_1. We then add 5.99 and A_1, saving the result in A_1. We multiply 6.99 and 1.06, saving the answer in A_2, and finish by adding A_1 and A_2, leaving the final answer in A_1. We can write this sequence of operations as follows:

$$4.99\ 1.06 * 5.99 + 6.99\ 1.06\ * +$$

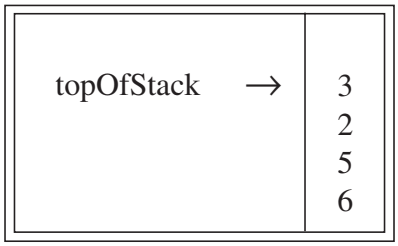
This notation is known as **postfix**, or **reverse Polish notation**, and is evaluated exactly as we have described above. The easiest way to do this is to use a stack. When a number is seen, it is pushed onto the stack; when an operator is seen, the operator is applied to the

two numbers (symbols) that are popped from the stack, and the result is pushed onto the stack. For instance, the postfix expression

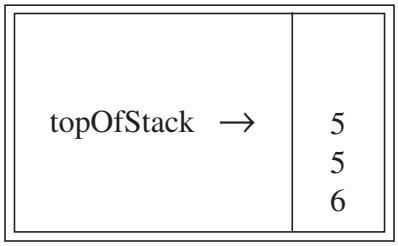
$$6\ 5\ 2\ 3 + 8 * + 3 + *$$

is evaluated as follows:

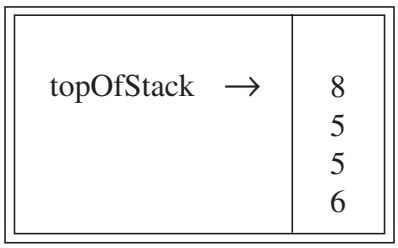
The first four symbols are placed on the stack. The resulting stack is



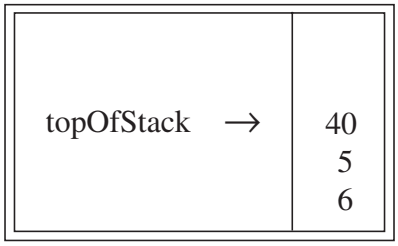
Next, a '+' is read, so 3 and 2 are popped from the stack, and their sum, 5, is pushed.



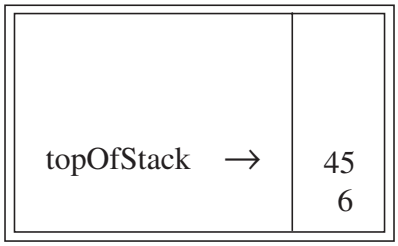
Next, 8 is pushed.



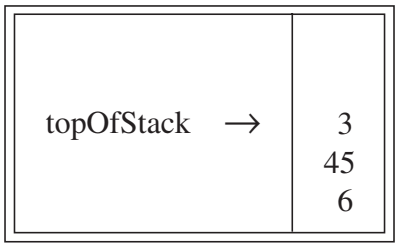
Now a '*' is seen, so 8 and 5 are popped, and $5 * 8 = 40$ is pushed.



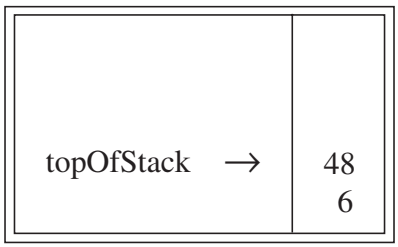
Next, a '+' is seen, so 40 and 5 are popped, and $5 + 40 = 45$ is pushed.



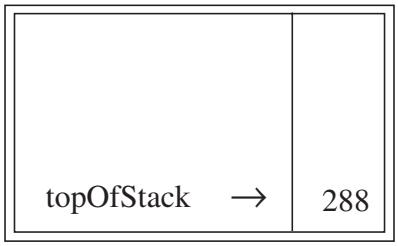
Now, 3 is pushed.



Next, '+' pops 3 and 45 and pushes $45 + 3 = 48$.



Finally, a '$*$' is seen and 48 and 6 are popped; the result, $6 * 48 = 288$, is pushed.



The time to evaluate a postfix expression is $O(N)$, because processing each element in the input consists of stack operations and therefore takes constant time. The algorithm to do so is very simple. Notice that when an expression is given in postfix notation, there is no need to know any precedence rules; this is an obvious advantage.

Infix to Postfix Conversion

Not only can a stack be used to evaluate a postfix expression, but we can also use a stack to convert an expression in standard form (otherwise known as **infix**) into postfix. We will concentrate on a small version of the general problem by allowing only the operators `+`, `*`, `(`, `)`, and insisting on the usual precedence rules. We will further assume that the expression is legal. Suppose we want to convert the infix expression

```
a + b * c + ( d * e + f ) * g
```

into postfix. A correct answer is `a b c * + d e * f + g * +`.

When an operand is read, it is immediately placed onto the output. Operators are not immediately output, so they must be saved somewhere. The correct thing to do is to place operators that have been seen, but not placed on the output, onto the stack. We will also stack left parentheses when they are encountered. We start with an initially empty stack.

If we see a right parenthesis, then we pop the stack, writing symbols until we encounter a (corresponding) left parenthesis, which is popped but not output.

If we see any other symbol (`+`, `*`, `(`), then we pop entries from the stack until we find an entry of lower priority. One exception is that we never remove a `(` from the stack except when processing a `)`. For the purposes of this operation, `+` has lowest priority and `(` highest. When the popping is done, we push the operator onto the stack.

Finally, if we read the end of input, we pop the stack until it is empty, writing symbols onto the output.

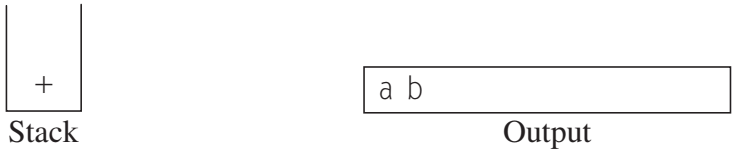
The idea of this algorithm is that when an operator is seen, it is placed on the stack. The stack represents pending operators. However, some of the operators on the stack that have high precedence are now known to be completed and should be popped, as they will no longer be pending. Thus prior to placing the operator on the stack, operators that are on the stack, and which are to be completed prior to the current operator, are popped. This is illustrated in the following table:

Expression	Stack When Third Operator Is Processed	Action
`a*b-c+d`	`-`	`-` is completed; `+` is pushed
`a/b+c*d`	`+`	Nothing is completed; `*` is pushed
`a-b*c/d`	`- *`	`*` is completed; `/` is pushed
`a-b*c+d`	`- *`	`*` and `-` are completed; `+` is pushed

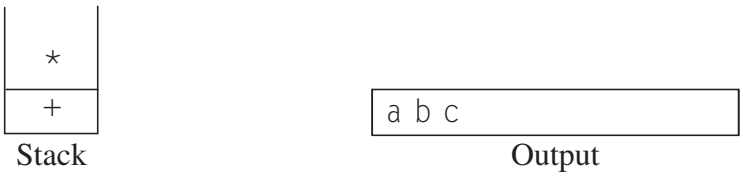
Parentheses simply add an additional complication. We can view a left parenthesis as a high-precedence operator when it is an input symbol (so that pending operators remain pending) and a low-precedence operator when it is on the stack (so that it is not accidentally removed by an operator). Right parentheses are treated as the special case.

To see how this algorithm performs, we will convert the long infix expression above into its postfix form. First, the symbol `a` is read, so it is passed through to the output.

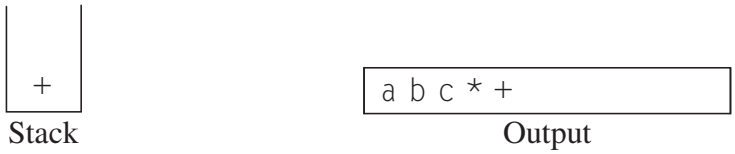
Then + is read and pushed onto the stack. Next b is read and passed through to the output. The state of affairs at this juncture is as follows:



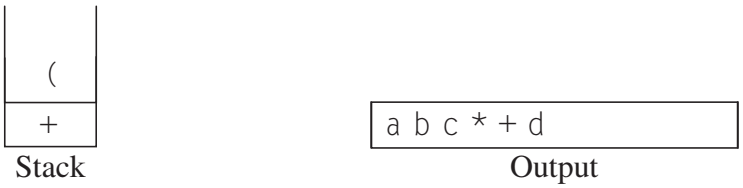
Next, a * is read. The top entry on the operator stack has lower precedence than *, so nothing is output and * is put on the stack. Next, c is read and output. Thus far, we have



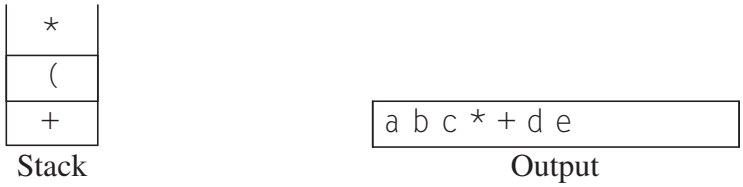
The next symbol is a +. Checking the stack, we find that we will pop a * and place it on the output; pop the other +, which is not of *lower* but equal priority, on the stack; and then push the +.



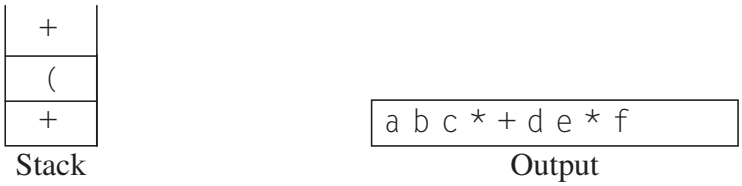
The next symbol read is a (. Being of highest precedence, this is placed on the stack. Then d is read and output.



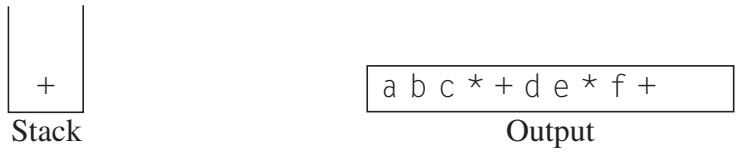
We continue by reading a *. Since open parentheses do not get removed except when a closed parenthesis is being processed, there is no output. Next, e is read and output.



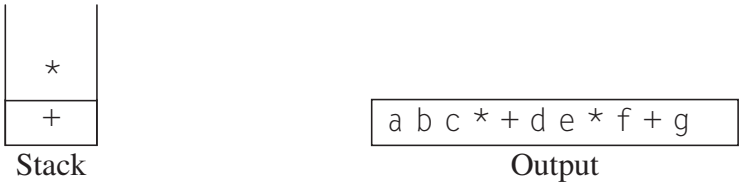
The next symbol read is a +. We pop and output * and then push +. Then we read and output f.



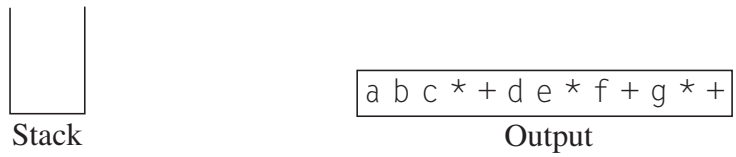
Now we read a), so the stack is emptied back to the (. We output a +.



We read a * next; it is pushed onto the stack. Then g is read and output.



The input is now empty, so we pop and output symbols from the stack until it is empty.



As before, this conversion requires only $O(N)$ time and works in one pass through the input. We can add subtraction and division to this repertoire by assigning subtraction and addition equal priority and multiplication and division equal priority. A subtle point is that the expression a - b - c will be converted to a b - c - and not a b c - -. Our algorithm does the right thing, because these operators associate from left to right. This is not necessarily the case in general, since exponentiation associates right to left: $2^{2^3} = 2^8 = 256$, not $4^3 = 64$. We leave as an exercise the problem of adding exponentiation to the repertoire of operators.

Function Calls

The algorithm to check balanced symbols suggests a way to implement function calls in compiled procedural and object-oriented languages. The problem here is that when a call is made to a new function, all the variables local to the calling routine need to be saved by the system, since otherwise the new function will overwrite the memory used by the calling routine's variables. Furthermore, the current location in the routine must be saved